

스테이션(PC) 라우팅 · 구동 코드 가이드

로버에 연결해 실행하는 코드는 무엇이고, 어떻게 실행하며, .py와 .ino가 어떻게 연결되는가

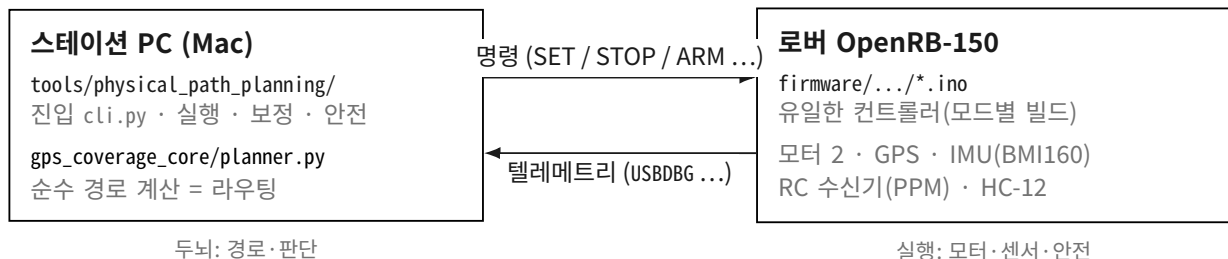
저장소 github.com/seonukkim/gps_hc12_robot

대상 OpenRB-150 로버 + HC-12 + GPS/IMU

- | | |
|-----------------|---------------------|
| 1 전체 구조 | 5 파이썬 ↔ 아두이노 연결 |
| 2 두 가지 운용 방식 | 6 실행 순서 (복붙용 명령) |
| 3 설치와 실행 · 모드 표 | 7 라우팅 코드만 원할 때 · 참고 |
| 4 파일별 역할 | |

한 줄 요약. 로버 안에서 도는 코드는 `firmware/openrb_robot_controller/openrb_robot_controller.ino` 하나. PC(스테이션)의 “라우팅 파이썬”은 `tools/physical_path_planning/` 패키지(진입점 `cli.py`)이고, 순수 경로 계산은 `gps_coverage_core/planner.py`에 있다. PC와 로버는 USB 시리얼 케이블 하나(`/dev/ttyACM0`, 115200 baud)로 연결된다.

1 전체 구조



연결 = USB 시리얼(`/dev/ttyACM0` · 115200) 한 줄. PC→로버는 저수준 구동 명령, 로버→PC는 약 0.5초마다 USBDDBG 텔레메트리 한 줄.

역할 분리. 두뇌(경로 · 판단)는 **파이썬**, 안전 실행(모터 · 센서)은 **펌웨어(.ino)**. 모든 실행 요약에는 안전 플래그 `ready_for_full_path_following=false`가 강제된다(완전 자율이 아니라 감독 · 가드 구동).

2 두 가지 운용 방식

상대가 말한 “PC 라우팅 코드”는 보통 ① **유선**이다.

- | | |
|------------------------|--|
| ① 유선
두뇌 = PC | PC 파이썬이 경로 · 판단을 하고 USB로 실시간 구동 명령을 보낸다. 모드 <code>auto-relative-run / run / execute-plan</code> . USB를 뽑으면 데드맨이 즉시 정지(설계). |
| ② 무선
두뇌 = 로버 | PC로 온보드 패턴 펌웨어를 1회 업로드만 하면 이후 USB 없이 RC 송신기로 동작. 모드 <code>rc-auto-pattern</code> (꺾자 왕복). 이때 PC 파이썬은 빌드 · 업로드 도구로만 쓰인다. |

3 설치와 실행

3.1 설치 (PC, 최초 1회)

의존성 설치

```
uv sync --extra dev
# 또는
pip install -r requirements.txt
```

핵심 의존성: pyserial, pynmea2, numpy/scipy/pyproj/geographiclib, matplotlib. 펌웨어 업로드 모드는 arduino-cli + OpenRB-150 보드 코어가 필요.

3.2 모든 실행의 공통 진입점

디스패처 (실제 로직은 cli.py의 cmd_* 함수)

```
bash scripts/run_physical_path_planner.sh <mode> [options]
```

<mode> 는 아래 표의 모드 이름.

3.3 주요 모드

모드	모터	역할
diagnose	정지	연결 · 센서 확인 (텔레메트리만 요약)
gps-wait	정지	쓸 만한 GPS 픽스 대기
rc-input-diagnose	정지	RC 수신기 채널 입력 진단
manual-control	수동 (RC)	PPM 수동 조작 펌웨어 업로드 · 모니터
preview	정지	꺾자 커버리지 경로 계획 + PNG 렌더
inspect-plan	정지	저장된 플랜/이미지 확인
tune-motion	구동	대화형 모터 캘리브레이션
calibrate-turn	구동	회전 각도 캘리브레이션 (IMU yaw)
calibration-check	정지	캘리브레이션 완성도 점검
align-heading	구동	첫 레인 방향으로 로버 정렬
run / execute-plan	구동	경로를 유선으로 실행
auto-relative-run	구동	AUTO 스위치 대기 후 상대경로 1회 유선 실행
rc-auto-pattern	구동	무선 온보드 꺾자 패턴 펌웨어 업로드

산출물 (트레이스 CSV · 요약 JSON · PNG)은 outputs/ 아래에 저장되며 깃에 커밋하지 않는다.

4 파일별 역할

임포트 방향은 한 방향 (리프 먼저): geometry, calibration, telemetry, safety, checks → executor → controller → cli.

4.1 tools/physical_path_planning/ — PC 실행·구동의 본체

파일	역할
cli.py	사용자 CLI · 시리얼 오픈 · 펌웨어 빌드/업로드 · 요약(JSON) 기록 (가장 큼)
geometry.py	꺾자 경로 기하 생성. 코너를 회전→스텝→회전으로 분해
calibration.py	전진/후진/회전 캘리브레이션 값 · 회전 실제각(target_angle_deg)
telemetry.py	USBDBG 라인 파싱 + GPS/IMU/RC/모터 필드 접근자
executor.py	시리얼에 명령 쓰기 · 가드 펄스 FSM(ARM→ACK→완료→STOP)
controller.py	경로 실행 감독 루프(stop_correct_go): 구동→정지→읽기→보정 반복
alignment.py	초기 헤딩 정렬(GPS 변위 프로브 + IMU 제자리 회전)
tuning.py	대화형 캘리브레이션 후보 조정 · 승인 · 백업/리셋
preview.py	계획 경로를 PNG로 렌더(현장에서 눈으로 확인)
safety.py, checks.py	ACK/STOP · 출력0 확인 · ready_for_full_path_following=false 강제

4.2 gps_coverage_core/ — 순수 경로 계산(하드웨어 비의존)

파일	역할
planner.py	위경도 → 미터 변환, 레인 오프셋, 왕복 웨이포인트 — 라우팅의 심장
protocol.py	PC → 로버 명령 프레임 · XOR 체크섬 · 값 클램프
geo.py	거리·방위 계산
nmea.py	GPS NMEA 파싱
imu.py	IMU 계산 헬퍼
side_tool_planner.py	보조 도구(사이드 툴) 경로

4.3 펌웨어와 ROS2

- firmware/openrb_robot_controller/openrb_robot_controller.ino — 로버의 유일한 컨트롤러. 하나의 파일을 컴파일 -D 플래그로 여러 모드로 빌드 (수동 조작 / 가드 펄스 / 무선 패턴). PPM 디코드 · GPS · IMU · 모터 · 안전게이트 · USBDBG 회신.
- firmware/*_probe, *_test — GPS/IMU/PPM 배선 점검용 일회성 진단 스케치(운용과 무관).
- ros2_ws/ — ROS2 노드. **현재 스켈레톤이라 지금 전달할 라우팅 코드에는 불포함.**

5 파이썬 ↔ 아두이노 연결

두 지점에서 연결된다.

5.1 빌드타임 — 파이썬이 펌웨어를 빌드·업로드

업로드가 필요한 모드에서 cli.py가 arduino-cli를 호출해 같은 .ino를 모드별 -D 플래그로 컴파일·업로드한다.

예: 무선 패턴 빌드 플래그

```
arduino-cli compile --fqbn OpenRB-150:samd:OpenRB-150 \
  --build-property "compiler.cpp.extra_flags=-DRC_AUTO_PATTERN=1 -DMANUAL_CONTROL_PPM=1 -DIMU_ENABLE
  ↪ =1 ..." \
  firmware/openrb_robot_controller
```

즉 “어떤 파이썬 모드로 올렸나”가 “로버가 어떤 펌웨어 모드로 도나”를 결정한다.

5.2 런타임 — USB 시리얼 문자열 프로토콜

포트/속도: /dev/ttyACM0 · 115200 baud (Mac은 /dev/tty.usbmodemXXXX). a(physical A)는 전/후진, b(physical B)는 좌/우 조향이며 각각 $[-1, 1]$ 로 클램프되어 로버가 좌·우 바퀴로 믹싱한다.

PC → 로버 — 개행 종료 ASCII (executor.write_command 전송)

```
USB_DRIVE_LIVE_SET seq=N a=<fwd> b=<steer> ms=.. ttl=.. # 연속 구동 setpoint
USB_DRIVE_LIVE_STOP seq=N # 연속 구동 정지
USB_PULSE_TEST_ARM / USB_PULSE_TEST_STOP seq=N # 가드 펄스
STOP # 즉시 정지
```

첫 줄 USB_DRIVE_LIVE_SET이 실시간 구동의 핵심.

로버 → PC (약 0.5초마다 USBDBG 한 줄, 공백 구분 key=value)

텔레메트리 한 줄 예시

```
USBDBG mode=AUTO_RUNNING event=ACK rc_ok=true auto_sw=true
gps_sats=7 gps_hdop=1.2 imu_relative_yaw_deg=172.8
final_left_cmd=0.31 final_right_cmd=0.29 physical_output_active=true ...
```

telemetry.parse_usbdbg_rows가 dict로 파싱. event=ARM/ACK/STOP는 가드 펄스 상태 확인용.

6 실행 순서 (복붙용 명령)

로버 USB 연결 상태에서 위에서 아래로. 각 블록을 그대로 복사해 실행한다.

1 · 연결·센서 확인 — 모터 정지

```
bash scripts/run_physical_path_planner.sh diagnose \
  --out-dir outputs/physical_path_planning/diagnose
```

확인: USBDBG가 올라오고 GPS/IMU/RC 필드가 보이는지.

2 · GPS 픽스 대기 — 모터 정지

```
bash scripts/run_physical_path_planner.sh gps-wait \
  --timeout-s 300 --min-sats 5 --max-hdop 2.5 \
  --out-dir outputs/physical_path_planning/gps-wait
```

확인: gps_sats ≥ 5 , hdop가 임계값 이하로 안정.

3 · 경로 미리보기 — PNG 생성, 모터 정지

```
bash scripts/run_physical_path_planner.sh preview \
  --goal-mode relative_enu --goal-east-m 1.8 --goal-north-m 1.8 \
  --workspace-width-m 1.8 --step-spacing-m 0.6 \
  --out-dir outputs/physical_path_planning/preview
```

확인: 경로를 눈으로 먼저(preview_*.png).

4 · (필요 시) 모터 캘리브레이션 — 구동

```
bash scripts/run_physical_path_planner.sh tune-motion \
  --primitive forward \
  --out-dir outputs/physical_path_planning/tune_forward
```

전진/후진/회전 각각(회전은 calibrate-turn).

5 · 첫 레인 방향 정렬 — 구동

```
bash scripts/run_physical_path_planner.sh align-heading \
  --out-dir outputs/physical_path_planning/align
```

GPS 변위 프로브 + IMU 제자리 회전.

6 · 유선 실행 — 상대경로 1회 — 구동

```
bash scripts/run_physical_path_planner.sh auto-relative-run \
  --goal-east-m 1.8 --goal-north-m 1.8 \
  --workspace-width-m 1.8 --step-spacing-m 0.6 \
  --out-dir outputs/physical_path_planning/auto_run
```

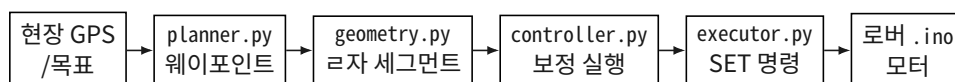
RC를 AUTO로 넘기면 현재 GPS를 원점으로 시작. 무선은 대신 rc-auto-pattern으로 1회 업로드 후 USB를 뽑고 RC 송신기로 운용.

7 라우팅 코드만 원할 때 · 참고

A. 순수 “경로 계산 로직”을 원하면 → 이 세 파일(우선순위 순):

1. gps_coverage_core/planner.py — 위경도 → 미터, 레인 오프셋, 왕복 웨이포인트(라우팅의 심장).
2. tools/physical_path_planning/geometry.py — 그 경로를 로버가 밟을 수 있는 ㄱ자 세그먼트(직진/코너 회전/스텝)로 분해.
3. tools/physical_path_planning/controller.py — 세그먼트를 GPS/IMU 피드백으로 보정하며 실행하는 감속 루프.

B. “구동 인터페이스”를 원하면 → tools/physical_path_planning/executor.py + gps_coverage_core/protocol.py (§ 5 시리얼 프로토콜).

데이터 흐름**참고 문서 (docs/)**

문서	내용
README_physical_path_planning.md	현장 실행 런북(유선/무선)
physical_path_planning_architecture.md	모듈 아키텍처
station_routing_code_guide.md	이 문서의 원본 텍스트(전체 상세)
claude_collaboration_guide.md	제어 의미론 · 트러블슈팅 결정 트리
protocol.md, wiring.md, rc_channel_map.md	프로토콜 · 배선 · RC 채널

안전. 완전 자율이 아니라 감속·가드 구동. 실제 주행 전 preview로 경로를 확인하고, GPS 픽스·캘리브레이션을 먼저 끝낸다.
 재생성 xelatex station_routing_code_guide.tex